

TaskFlow API

POC de Testes de Integração com IA

Versão 1.0.0

Gerado em: 22/06/2026

Douglas Tertuliano + Claude Code (Opus 4.6)

Introdução

O Problema

Em projetos de longa duração com APIs escaláveis, o maior custo de manutenção não é escrever o código — é manter a suíte de testes confiável enquanto a API evolui. Rotas mudam, regras de negócio crescem, integrações externas aparecem. Sem uma estratégia clara, os testes ficam para trás ou viram obstáculo.

Esta POC nasceu de uma pergunta estratégica do time:

"Como vamos testar nossas APIs no futuro — tanto local quanto em deploy — à medida que o projeto cresce?"

A resposta tradicional envolve containers Docker, bancos de dados reais e pipelines complexas de CI/CD. Nós escolhemos um caminho diferente: **repositórios in-memory, zero dependências externas e inteligência artificial como parte ativa do fluxo de qualidade.**

O Que Você Vai Aprender

Ao longo deste documento, vamos percorrer toda a jornada de construção de uma API REST completa — do scaffold inicial ao ebook que você está lendo agora. Cada capítulo cobre uma dimensão do projeto:

1. **Arquitetura** — como a separação em 3 camadas (Controller → Service → Repository) facilita os testes
2. **API e Rotas** — as 23 rotas implementadas, suas regras de negócio e exemplos de uso
3. **Estratégia de Testes** — a pirâmide de testes adotada, o padrão `createTestApp()` e como tudo roda no CI/CD
4. **IA nos Testes** — como o Claude Code CLI foi usado para gerar testes a partir de prompts conversacionais
5. **Conclusão** — métricas finais, lições aprendidas e próximos passos para produção

Stack Técnica

A escolha de cada ferramenta foi guiada por um princípio: **simplicidade operacional**. Nada que exija instalação de sistema, containers ou configuração manual de serviços.

Camada	Ferramenta	Justificativa
--------	------------	---------------

Runtime	Node.js 20 LTS + TypeScript 5	Padrão de mercado, tipagem estática
Framework HTTP	Express 4	Estável, amplamente documentado
Validação	Zod	Integração nativa com TypeScript
Test Runner	Vitest	ESM-native, mais rápido que Jest
Testes HTTP	Supertest	Padrão para integração em Node.js
Mocking externo	MSW v2	Intercepta requests sem adapters
Dados de teste	@faker-js/faker	Fixtures realistas e repetíveis
IA para testes	Claude Code CLI	Geração de testes via prompts
Docs API	Swagger UI	OpenAPI 3.0 documentado inline
PDF	md-to-pdf	Geração do ebook final

Como Navegar

Este documento foi projetado para ser lido em ordem, mas cada capítulo é autossuficiente. Se você já conhece a arquitetura, pule direto para o **Capítulo 3** (Estratégia de Testes). Se quer ver os resultados da IA, vá ao **Capítulo 4**.

O código-fonte completo está disponível no repositório. Cada decisão arquitetural está documentada como ADR (Architecture Decision Record) no arquivo

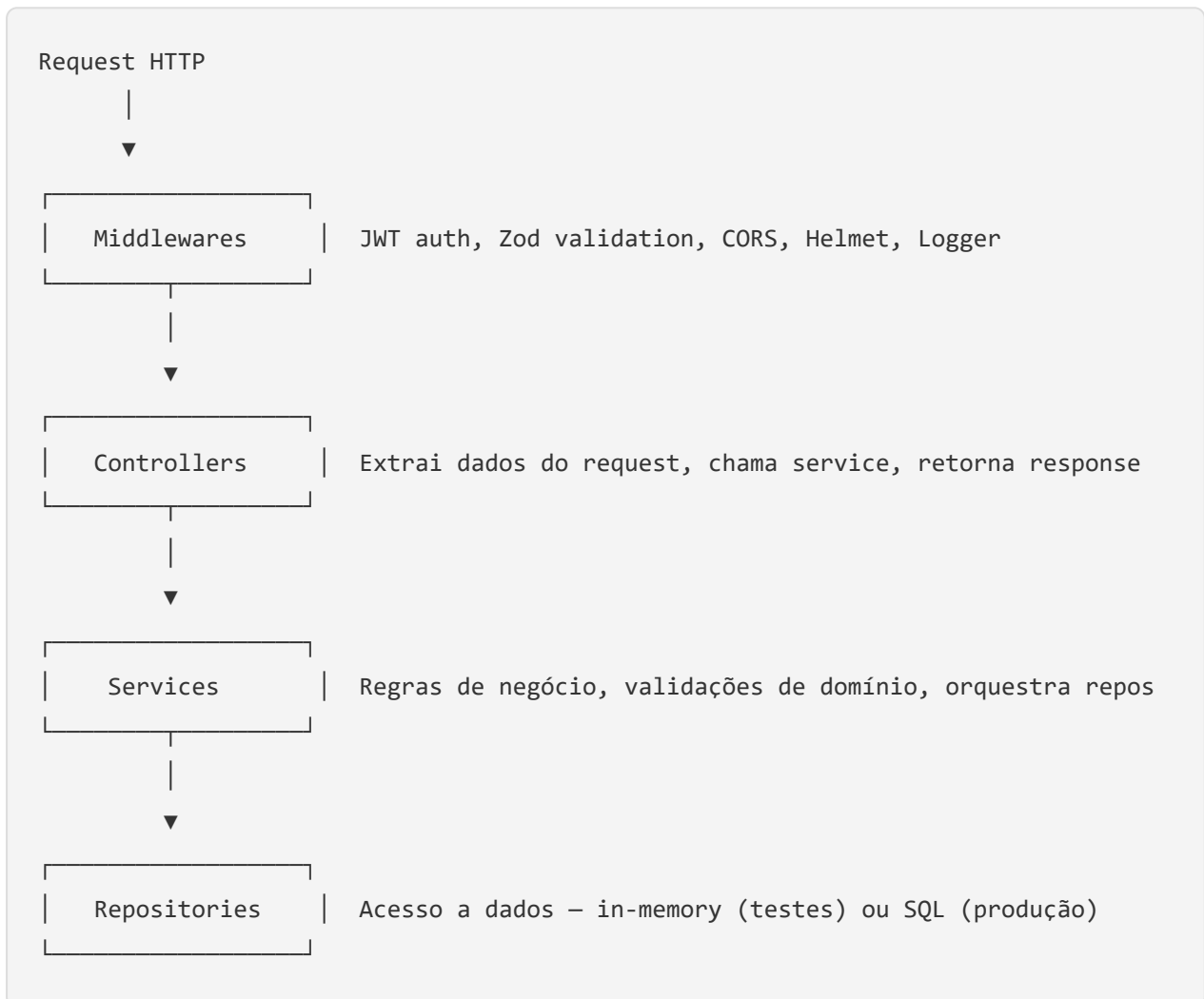
`docs/architecture/decisions.md` .

Domínio fictício (TaskFlow) escolhido para maximizar variedade de rotas, regras de negócio e casos de teste. A estratégia apresentada é aplicável a qualquer API REST em Node.js.

Arquitetura

Visão Geral

A TaskFlow API segue uma arquitetura em 3 camadas com injeção de dependência. Cada camada tem uma responsabilidade clara e não conhece os detalhes de implementação das camadas adjacentes.



Por Que Cada Camada Existe

Controllers

Os controllers são a fronteira HTTP da aplicação. Eles extraem dados de `req.params`, `req.body` e `req.query`, delegam para o service correspondente e formatam a resposta usando helpers padronizados (`success()`, `created()`, `noContent()`).

Regra fundamental: nenhuma lógica de negócio vive aqui. Se um controller precisa de um `if` que não seja para extrair dados, o código pertence ao service.

Services

Os services concentram todas as regras de negócio. O `TaskService.updateStatus()`, por exemplo, valida se a transição de status é permitida consultando `VALID_STATUS_TRANSITIONS` antes de persistir a mudança. O `ProjectService.delete()` verifica se existem tasks ativas antes de permitir a exclusão.

Regra fundamental: services não importam nada do Express. Seus parâmetros são DTOs tipados, não `req / res`.

Repositories

Os repositories abstraem o acesso a dados por trás de uma interface genérica `IRepository<T>`. Na POC, a implementação usa `Map<string, T>` para armazenamento in-memory. Em produção, a mesma interface seria implementada com queries SQL ou um ORM.

Regra fundamental: cada implementação de repository é substituível. O mesmo código de service roda com dados in-memory nos testes e com PostgreSQL em produção.

Injeção de Dependência

Todas as camadas recebem suas dependências via construtor. A função `createApp()` em `src/app.ts` é a raiz da composição:

```
createApp(repositories?)
├─ cria Services com os repositories
│   └─ cria Controllers com os services
│       └─ registra rotas com os controllers
```

Nos testes, `createTestApp()` chama `createApp()` com repositórios frescos a cada invocação — garantindo isolamento total entre testes.

Tratamento de Erros

A aplicação define uma hierarquia de erros customizados que mapeia diretamente para códigos HTTP:

Classe	Status	Uso
<code>NotFoundError</code>	404	Recurso não encontrado
<code>ValidationError</code>	400	Dados inválidos (com lista de erros)
<code>UnauthorizedError</code>	401	Autenticação ausente ou inválida
<code>ForbiddenError</code>	403	Permissão insuficiente
<code>ConflictError</code>	409	Duplicata ou conflito de estado

O middleware `errorHandler` captura qualquer erro lançado nas camadas inferiores, identifica o tipo e retorna a resposta HTTP apropriada com formato padronizado.

Decisões Arquiteturais (ADRs)

As decisões mais relevantes estão documentadas como ADRs no repositório:

- **ADR-001:** Repositórios in-memory em vez de Docker — elimina dependências externas para rodar testes, com a contrapartida de não testar queries SQL reais.
- **ADR-002:** Vitest em vez de Jest — suporte nativo a ESM e TypeScript, API compatível com Jest, configuração mais simples.
- **ADR-003:** Express em vez de Fastify — familiaridade da comunidade, vasta documentação, foco da POC é em testes e não em performance.
- **ADR-004:** Estratégia local vs deploy — testes em camadas com `BASE_URL` configurável, os mesmos smoke tests rodam contra localhost ou staging.
- **ADR-005:** Smoke tests leves — ~10 testes E2E cobrindo o caminho crítico em vez de duplicar toda a suíte de integração.

Cada ADR segue o formato: Contexto → Decisão → Consequências (positivas e negativas).

API e Rotas

Mapa Completo de Rotas

A TaskFlow API expõe 23 rotas organizadas em 4 domínios. Todas as rotas (exceto autenticação e health check) requerem um token JWT no header `Authorization: Bearer <token>`.

Método	Path	Auth	Descrição
GET	/health	Não	Health check
POST	/auth/register	Não	Registrar novo usuário
POST	/auth/login	Não	Autenticar e obter token
GET	/users	Sim	Listar usuários
GET	/users/:id	Sim	Buscar usuário por ID
PUT	/users/:id	Sim	Atualização completa
PATCH	/users/:id	Sim	Atualização parcial
DELETE	/users/:id	Sim	Deletar usuário
GET	/projects	Sim	Listar projetos (filtro: <code>?status=</code>)
GET	/projects/:id	Sim	Buscar projeto por ID
POST	/projects	Sim	Criar projeto
PUT	/projects/:id	Sim	Atualização completa
PATCH	/projects/:id	Sim	Atualização parcial
DELETE	/projects/:id	Sim	Deletar projeto
GET	/tasks	Sim	Listar tasks (filtros: <code>?projectId=</code> , <code>?status=</code> , <code>?priority=</code>)

GET	/tasks/:id	Sim	Buscar task por ID
POST	/tasks	Sim	Criar task
PUT	/tasks/:id	Sim	Atualização completa
PATCH	/tasks/:id/status	Sim	Transição de status
DELETE	/tasks/:id	Sim	Deletar task
GET	/tasks/:id/comments	Sim	Listar comentários
POST	/tasks/:id/comments	Sim	Adicionar comentário
DELETE	/tasks/:id/comments/:commentId	Sim	Deletar comentário

Exemplos de Request/Response

Registrar usuário

```
POST /auth/register
Content-Type: application/json
```

```
{
  "name": "Maria Silva",
  "email": "maria@example.com",
  "password": "senha123"
}
```

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs... ",
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Maria Silva",
      "email": "maria@example.com",
      "role": "MEMBER",
      "createdAt": "2026-06-17T10:00:00.000Z"
    }
  }
}
```



```
}
}
```

Criar task e transicionar status

POST /tasks

Authorization: Bearer <token>

```
{ "projectId": "abc-123", "title": "Implementar login", "priority": "HIGH" }
→ 201 Created (status: TODO)
```

PATCH /tasks/task-id/status

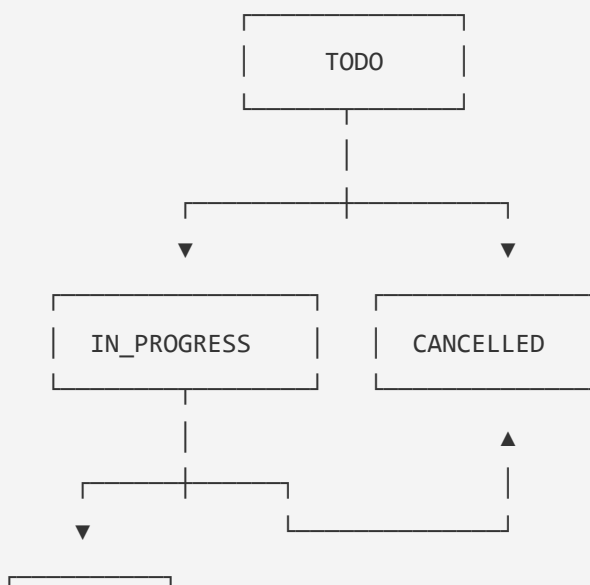
Authorization: Bearer <token>

```
{ "status": "IN_PROGRESS" }
→ 200 OK (status: IN_PROGRESS)
```

Regras de Negócio

Máquina de estados das Tasks

A rota `PATCH /tasks/:id/status` é a mais rica em regras de negócio. As transições de status seguem um fluxo definido:



DONE

- **TODO** → **IN_PROGRESS**: permitido
- **IN_PROGRESS** → **DONE**: permitido
- **TODO** ou **IN_PROGRESS** → **CANCELLED**: permitido
- **TODO** → **DONE**: bloqueado (deve passar por IN_PROGRESS)
- **DONE** → **qualquer**: bloqueado (estado terminal)
- **CANCELLED** → **qualquer**: bloqueado (estado terminal)

Transições inválidas retornam `400 Bad Request` com mensagem descritiva.

Validações de exclusão

- **Deletar projeto**: bloqueado se existem tasks com status TODO ou IN_PROGRESS (retorna 409)
- **Deletar usuário**: bloqueado se o usuário tem tasks IN_PROGRESS (retorna 403)
- **Deletar task**: bloqueado se o status é IN_PROGRESS (retorna 409)
- **Deletar comentário**: apenas o autor pode deletar (retorna 403 para outros)

Validação de dados

Todas as rotas com body usam schemas Zod para validação. Campos obrigatórios ausentes, tipos incorretos ou valores fora do enum retornam `400 Bad Request` com a lista de erros de validação.

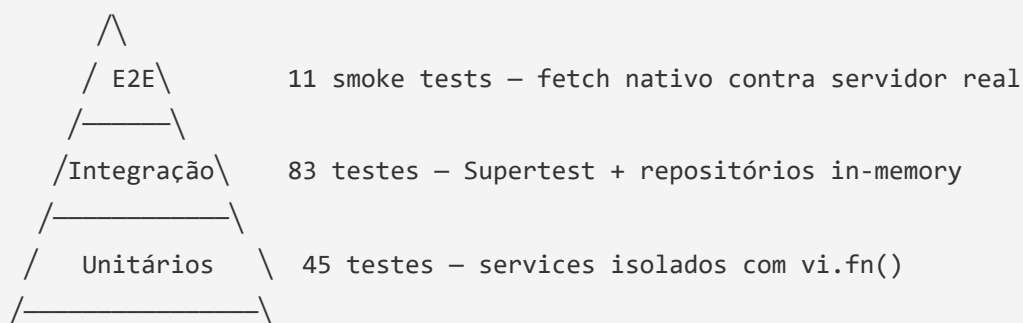
Documentação interativa

A API disponibiliza uma interface Swagger UI em `GET /api-docs` com todos os endpoints documentados, schemas de request/response e a possibilidade de testar as rotas diretamente pelo navegador.

Estratégia de Testes

A Pirâmide de Testes

A TaskFlow API adota uma pirâmide de testes em 3 camadas, cada uma com responsabilidade e ferramentas distintas:



Camada	Quantidade	Ferramenta	Tempo	O que valida
Unitários	45	Vitest + vi.fn()	~2.6s	Lógica de negócio isolada
Integração	83	Supertest + in-memory	~8.8s	Rotas HTTP + middlewares + services
E2E	11	fetch nativo	~1s	Deploy real (rede, CORS, env vars)

Total: 139 testes em ~12 segundos, sem Docker, sem banco externo.

Por Que In-Memory

A decisão mais importante da estratégia é usar repositórios in-memory em vez de Docker com banco real. O impacto é significativo:

- **Setup zero:** `npm install` e pronto. Sem Docker Desktop, sem `docker-compose up`, sem aguardar containers.
- **Velocidade:** 139 testes em 12 segundos. Com banco real, seriam minutos.
- **CI/CD trivial:** o pipeline do GitHub Actions não precisa de service containers.

- **Isolamento perfeito:** cada teste cria seus próprios repositórios via `createTestApp()` .

A contrapartida é clara: não testamos queries SQL, índices ou constraints do banco. Para produção, recomendamos adicionar uma camada de testes com Testcontainers.

O Padrão `createTestApp()`

O coração da infraestrutura de testes é a função `createTestApp()` :

```
function createTestApp(): { app: Express; repositories: Repositories }
```

Cada `describe` ou `it` cria sua própria instância do app com repositórios completamente frescos. Isso elimina o problema mais comum em testes de integração: **estado compartilhado entre testes**.

```
describe('GET /projects', () => {  
  it('deve retornar lista vazia', async () => {  
    const { app } = createTestApp(); // app limpo  
    const res = await request(app).get('/projects');  
    expect(res.body.data).toHaveLength(0);  
  });  
});
```

Data Factories

O módulo `data-factory.ts` usa `@faker-js/faker` com locale `pt_BR` para gerar dados de teste realistas:

- `makeUser()` — gera um DTO de criação de usuário com nome, email e senha
- `makeProject()` — gera um DTO de projeto com nome e descrição
- `makeTask()` — gera um DTO de task com título e prioridade
- `makeUserSeed()` — cria e persiste um usuário diretamente no repositório

As funções `seed` são úteis quando o teste precisa de dados pré-existentis sem passar pela API (ex: criar um owner antes de testar a criação de projetos).

MSW para Serviços Externos

O Mock Service Worker (MSW v2) intercepta requests HTTP a serviços externos nos testes:

- **Serviço de email:** `POST https://notifications.taskflow.io/send` retorna 202 com `messageld`
- **ViaCEP:** `GET https://viacep.com.br/ws/:cep/json/` retorna dados de endereço
- **Fallback:** qualquer request HTTPS não mapeado retorna 500 + log de warning

O setup no Vitest garante que handlers são resetados entre testes (`afterEach: server.resetHandlers()`).

CI/CD com GitHub Actions

O pipeline em `.github/workflows/ci.yml` executa em 3 jobs:

1. **test** — `npm run test` + `npm run test:integration` + upload de cobertura
2. **build** — `npm run build` (compilação TypeScript)
3. **e2e** — inicia o servidor, aguarda health check, roda smoke tests

Variáveis de ambiente fixas no CI: `NODE_ENV=test` , `JWT_SECRET=test-secret-for-ci` .

Guia Prático: Como Escrever um Novo Teste

Para adicionar um teste de integração para uma nova rota:

1. Crie ou edite o arquivo em `tests/integration/`
2. Importe `createTestApp` , `makeUser` , `getAuthToken` dos helpers
3. No `describe` , crie o app: `const { app } = createTestApp()`
4. Obtenha um token: `const token = await getAuthToken(app)`
5. Faça o request: `const res = await request(app).get('/rota').set('Authorization', \ Bearer ${token})``
6. Valide: `expect(res.status).toBe(200)`

Para testar um cenário com dados pré-existentes, use as funções `seed` do data-factory para popular os repositórios antes do request.

IA nos Testes

A Abordagem

Esta POC utilizou o **Claude Code CLI** (Claude como agente de engenharia no terminal) como parte ativa do fluxo de qualidade. Em vez de escrever cada teste manualmente, usamos prompts conversacionais para gerar suítes completas de testes a partir das definições de rota e regras de negócio.

O fluxo foi:

1. **Construir a API primeiro** (Fases 0 e 1) — modelos, repositórios, services, controllers, rotas
2. **Criar a infraestrutura de testes** (Fase 2) — helpers, factories, MSW
3. **Gerar os testes via IA** (Fase 3) — prompts descrevendo os cenários esperados

Exemplos de Prompts

Prompt para testes unitários dos services

O prompt descrevia os cenários esperados por service — por exemplo, para o `TaskService` :

"Crie testes unitários para o `TaskService`. Cubra: create em projeto `ACTIVE`, create em projeto `ARCHIVED` (`ConflictError`), `updateStatus` com transição válida `TODO` → `IN_PROGRESS`, transição inválida `TODO` → `DONE` (`ValidationError`), delete com status `IN_PROGRESS` (`ConflictError`). Use `vi.fn()` para mockar repositórios."

Resultado: 21 testes gerados, todos passando na primeira execução. O Claude identificou corretamente os mocks necessários, usou `it.each()` para testar múltiplas transições inválidas e nomeou cada teste em português conforme solicitado.

Prompt para testes de integração

Para as rotas de Projects, o prompt listava os cenários HTTP esperados:

"Crie testes de integração para `/projects`. `GET` retorna lista vazia, filtra por status, retorna 401 sem auth. `POST` cria com 201, retorna 400 sem campos obrigatórios. `DELETE` retorna 409 com tasks ativas."

Resultado: 19 testes gerados. Na primeira execução, 12 falharam por nomes de propriedade incorretos nos repositórios (`repos.userRepo` vs `repos.userRepository`). Após uma correção pontual, todos passaram.

Prompt para testes E2E

"Crie smoke tests E2E com fetch nativo contra servidor real. Health check, fluxo de auth, fluxo completo projeto → task → transições de status, proteção de rotas sem token."

Resultado: 11 testes gerados. Problema encontrado: a variável `BASE_URL` não era propagada para processos fork do Vitest no Windows. Corrigido adicionando `test.env` no config do Vitest.

O Que Funcionou Bem

Velocidade de geração. A Fase 3 (45 unitários + 83 integração + 11 E2E) foi executada em poucas interações. Escrever 139 testes manualmente levaria significativamente mais tempo.

Qualidade estrutural. Os testes gerados seguem boas práticas: `describe` aninhados por método/rota, nomes descritivos, uso correto de `beforeEach` / `afterEach` , assertions com `expect.objectContaining()` .

Cobertura de edge cases. Ao listar explicitamente os cenários no prompt, o Claude cobriu sistematicamente cada caso — incluindo cenários de erro e validação que frequentemente são esquecidos em testes manuais.

Consistência. Todos os testes seguem o mesmo padrão de organização, nomenclatura e uso dos helpers, criando uma suíte homogênea.

O Que Precisou de Ajuste

Nomes de propriedade. No teste de integração de Projects, o Claude usou `repos.userRepo` em vez de `repos.userRepository` . Isso acontece quando a IA infere nomes com base em convenções genéricas em vez de ler o código-fonte exato.

Propagação de variáveis de ambiente. O teste E2E no Windows não propagava `BASE_URL` para forks do Vitest. Embora o Claude tenha gerado o código funcional, o problema era de configuração do runtime — algo que requer execução real para detectar.

Comportamento da API vs expectativa. Alguns testes assumiam controle de ownership (MEMBER não pode editar outro user) que a API não implementa. Os testes foram ajustados para refletir o comportamento real.

Reflexão: Vantagens e Limitações

A IA é mais eficaz quando recebe contexto preciso. Prompts que listam os cenários esperados com verbos HTTP e status codes geram testes melhores do que prompts vagos como "teste tudo".

A IA não substitui a execução. Problemas de runtime (env vars no Windows, nomes de propriedade incorretos) só aparecem ao rodar os testes. A IA gera a estrutura; o desenvolvedor valida e ajusta.

Melhor caso de uso: geração em lote de testes para APIs com rotas CRUD padronizadas, onde os cenários são repetitivos (200, 400, 401, 404, 409). A IA elimina o trabalho mecânico e permite ao desenvolvedor focar nos cenários de negócio que realmente importam.

Conclusao e Proximos Passos

O Que Foi Demonstrado

Esta POC comprovou que é viável construir e manter uma suíte de testes abrangente para APIs Node.js **sem dependências externas pesadas** e **com auxílio de inteligência artificial**.

Resultados Alcançados

Metrica	Valor
Rotas na API	23
Testes unitarios	45
Testes de integracao	83
Testes E2E (smoke)	11
Total de testes	139
Cobertura de codigo (services)	67.79% statements, 93.54% branches
Tempo de execucao (unitarios)	~2.6s
Tempo de execucao (integracao)	~8.8s
Tempo de execucao (E2E)	~1s
Dependencias externas para testes	0 (zero Docker, zero banco)

Pilares Validados

1. **Arquitetura testavel.** A separacao em 3 camadas com injecao de dependencia permite substituir repositorios reais por in-memory sem alterar nenhum service ou controller.
2. **Testes rapidos e confiaveis.** 139 testes em ~12 segundos, sem flakiness causada por estado compartilhado ou timeouts de rede.

3. **CI/CD simples.** O pipeline do GitHub Actions roda unitarios, integracao e E2E sem service containers — apenas `npm ci` e `npm test`.
4. **IA como acelerador.** O Claude Code CLI gerou a maior parte dos 139 testes a partir de prompts descritivos, com taxa de aproveitamento superior a 90%.

Licoes Aprendidas

O que funcionou bem:

- Repositorios in-memory com `Map<string, T>` sao surpreendentemente adequados para testes de integracao de APIs REST
- O padrao `createTestApp()` elimina completamente problemas de estado compartilhado entre testes
- MSW v2 para mock de servicos externos e transparente — o codigo da aplicacao nao sabe que esta sendo interceptado
- Prompts estruturados (com cenarios e status codes esperados) produzem testes de alta qualidade via IA

O que mudaria numa proxima iteracao:

- Adicionar uma camada de testes com Testcontainers para validar queries SQL reais contra PostgreSQL
- Implementar contract tests (Pact) para validar integracao com servicos externos
- Aumentar cobertura dos services AuthService e CommentService (atualmente 0% em unitarios)
- Usar seed mais robusto nos testes E2E para nao depender de estado in-memory do servidor

Proximos Passos para Producao

1. Banco de dados real

Substituir os repositorios in-memory por implementacoes com PostgreSQL (via Knex ou Prisma). Manter os repositorios in-memory para testes e adicionar uma camada de testes com Testcontainers para validar queries SQL.

2. Testes de contrato

Adicionar Pact ou PactFlow para validar que os contratos entre a TaskFlow API e seus consumidores (frontend, mobile, outros servicos) se mantem estaveis conforme a API evolui.

3. Testes de performance

Introduzir k6 para load testing com cenários realistas: fluxo de autenticação, CRUD de projetos, transições de status de tasks sob carga.

4. Observabilidade nos testes

Integrar métricas de cobertura, tempo de execução e taxa de falha em um dashboard (Grafana, Datadog) para acompanhar a saúde da suite de testes ao longo do tempo.

5. Escalabilidade multi-serviço

Replicar a estratégia de testes para outros microserviços do ecossistema, mantendo o mesmo padrão de helpers, data factories e CI/CD. O padrão `createTestApp()` e as data factories são reutilizáveis como pacote interno.

POC executada por Douglas Tertuliano com auxílio do Claude Code (Opus 4.6) entre junho de 2026.